Python IP Allow List Automation

Project Type: Python Automation / Cybersecurity
Focus: File I/O, Data Processing, Access Control, Conditional Logic
Language: Python

Project Overview

I developed a Python algorithm that automates the process of maintaining an IP address allow list used to control access to restricted information. The program reads authorized IP addresses from a text file, compares them against a list of addresses that should no longer have access, removes identified matches, and updates the original file.

The project demonstrates how Python can be used to automate a repetitive security administration task while maintaining accurate access-control information.

Scenario

An organization maintains an `allow_list.txt` file containing IP addresses that are permitted to access restricted resources. As access requirements change, certain IP addresses need to be removed.

Manually reviewing the allow list each time an address needs to be removed can become inefficient and introduces the possibility of human error. I created an algorithm that programmatically processes the allow list against a predefined `remove_list` and updates the file so that only currently authorized IP addresses remain.

Algorithm

The algorithm follows a structured **input** → **process** → **output** workflow:

1. Open the file containing the authorized IP addresses.
2. Read the contents of the file into Python.
3. Convert the IP address data from a string into a list.
4. Iterate through the IP addresses designated for removal.
5. Determine whether each address exists in the allow list.
6. Remove matching IP addresses.

7. Convert the revised list back into a formatted string.
8. Write the updated information back to the original file.

This approach allows multiple IP addresses to be evaluated and removed automatically rather than requiring each entry to be manually located and deleted.

Technical Walkthrough

1. Opening the Allow List

I first stored the file name in the `import_file` variable and used Python's `with` statement and `open()` function to access the file.

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
```

Opening the file in `"r"` (read) mode allows its contents to be accessed without modifying them. The `with` statement also provides safe file handling by automatically closing the file after the operation is completed.

2. Reading the File

I used the `.read()` method to retrieve the contents of the allow list and store them in the `ip_addresses` variable.

```
with open(import_file, "r") as file:
    ip_addresses = file.read()
```

At this point, the contents of the file are represented as a string that Python can process.

3. Converting the Data Into a List

Because individual IP addresses needed to be evaluated, I converted the string into a list using `.split()`.

```
ip_addresses = ip_addresses.split()
```

This transformation makes each IP address an individual list element, allowing the algorithm to evaluate and manipulate specific addresses.

4. Iterating Through the Remove List

I created a `for` loop to process each IP address contained in `remove_list`.

```
for element in remove_list:
```

During each iteration, `element` represents one IP address scheduled for removal. This allows the same comparison logic to be automatically applied to every address in the removal list.

5. Identifying and Removing Matches

Inside the loop, I implemented conditional logic to determine whether the current IP address exists in the allow list.

```
for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)
```

The `if` statement and `in` operator perform a membership check. When a match is found, `.remove()` deletes that IP address from `ip_addresses`.

Using `.remove()` is appropriate for this dataset because the allow list does not contain duplicate IP addresses. Therefore, each matching address only needs to be removed once.

6. Updating the Allow List File

After processing the remove list, I converted the updated list back into a string.

```
ip_addresses = "\n".join(ip_addresses)
```

Applying `.join()` to `"\n"` places each IP address on a separate line and prepares the information to be written back to the text file.

I then reopened the original file in `"w"` (write) mode and used `.write()` to replace its previous contents with the updated allow list.

```
with open(import_file, "w") as file:
    file.write(ip_addresses)
```

This final operation persists the changes so that the file reflects the revised access permissions.

Complete Algorithm

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)

ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

Skills Demonstrated

This project demonstrates practical experience with several foundational Python and automation concepts, including:

- **Python file I/O:** Reading from and writing to external files using `open()`, `.read()`, and `.write()`.
- **Context managers:** Using `with` statements for safe and efficient file handling.
- **Data transformation:** Converting information between strings and lists with `.split()` and `.join()`.
- **Iteration:** Processing multiple records automatically using `for` loops.
- **Conditional logic:** Using `if` statements to make decisions based on data.
- **Membership testing:** Using the `in` operator to identify matching values.
- **List manipulation:** Programmatically removing specific elements with `.remove()`.
- **Automation:** Replacing a repetitive manual access-management process with reusable code.
- **Security fundamentals:** Applying automation to an IP-based access-control scenario.

Security & Business Impact

The algorithm demonstrates how relatively simple Python automation can support security operations. Automating the comparison between authorized and unauthorized IP addresses

reduces the amount of manual work required to maintain an access list and helps ensure that identified addresses are consistently removed.

From a business perspective, this type of automation can improve operational efficiency, reduce opportunities for manual errors, and provide a repeatable process for maintaining access-control data.

The same programming concepts could also be extended to larger workflows involving log analysis, user access management, configuration files, data validation, and security operations.

Key Takeaways

This project strengthened my understanding of how Python can be used to solve practical operational and security problems. Rather than treating the file as static information, I created a workflow that **ingests data, transforms it, applies decision-making logic, modifies the dataset, and persists the results**.

One of my biggest takeaways was understanding how individual Python concepts work together as part of a complete automation workflow. File handling, lists, loops, conditional statements, and methods each perform relatively simple operations independently, but combining them creates a useful algorithm capable of automating a real-world task.

The project also reinforced an important principle of automation: repetitive processes that follow consistent rules are strong candidates for programmatic solutions.